



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2413 — Bases de Datos — Sección 3 — 2'2026

PAUTA TAREA 1

Calculo de la nota de la tarea

Cada Parte 1, 2, 3 y 4 tiene una nota T_1 , T_2 , T_3 y T_4 y el informe tiene una nota I . La nota final de la tarea se calcula como:

$$\text{Nota Tarea 1} = \frac{1,25 \cdot T_1 + 1,25 \cdot T_2 + 1,25 \cdot T_3 + 1,25 \cdot T_4 + I}{6}$$

Parte 1

Cada tabla tiene un puntaje de 1 punto (en total sumando 8 puntos), en caso de que alguna tabla de un estudiante no sea igual a la de la solución, obtiene 0 puntos en esa tabla.

Debido a su extensión, la solución propuesta estará en el GitHub del curso.

Parte 2

Cada consulta de la solución tiene un puntaje de **1 punto** (en total sumando 6 puntos). Para cada consulta el puntaje se divide en:

- 1 punto: Tabla resultante correcta.
- 0.8 puntos: Tabla entregada en un orden distinto al pedido (según ORDER BY).
- 0 puntos: Tabla incorrecta.

Y específicamente para las 2 consultas difíciles, podía ocurrir un problema de fan out si no se trataban con cuidado los JOINS, en este caso:

- 0.9 puntos: Tabla con error de fan out, pero el resto es correcto.

A continuación se presentaran las soluciones de las consultas. **IMPORTANTE:** tener en cuenta que son soluciones propuestas y hay múltiples formas de construir consultas SQL, la revisión no exige tener la misma sintaxis o uso de operadores, sino que la tabla resultante sea igual.

- Consulta facil_1:

```
SELECT
  o.order_id,
  c.customer_city,
  c.customer_state
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.customer_id
```

```
WHERE o.order_status = 'delivered'
AND c.customer_state = 'RS';
```

■ Consulta facil_2:

```
SELECT
    oi.order_id,
    oi.product_id,
    oi.price,
    p.product_category_name
FROM order_items AS oi
JOIN products AS p ON oi.product_id = p.product_id
WHERE oi.price = 151;
```

■ Consulta intermedio.1:

```
SELECT
    c.customer_state,
    COUNT(o.order_id) AS entregas_atrasadas
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.order_id IN (
    SELECT order_id
    FROM orders
    WHERE order_status = 'delivered'

    INTERSECT

    SELECT order_id
    FROM orders
    WHERE order_delivered_customer_date > order_estimated_delivery_date
)
GROUP BY c.customer_state
ORDER BY entregas_atrasadas DESC;
```

■ Consulta intermedio.2:

```
SELECT
    s.seller_state as estado_origen,
    c.customer_state as estado_destino,
    COUNT(oi.order_item_id) as total_articulos_enviados,
    AVG(oi.freight_value) as envio_promedio
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
JOIN customers c ON o.customer_id = c.customer_id
JOIN sellers s ON oi.seller_id = s.seller_id
WHERE s.seller_state != c.customer_state
GROUP BY s.seller_state, c.customer_state
ORDER BY total_articulos_enviados DESC;
```

■ Consulta difícil_1:

```

SELECT
    gasto.customer_city,
    gasto.gasto_total,
    vendedores.vendedores_distintos
FROM (
    SELECT c.customer_city, SUM(op.payment_value) AS gasto_total
    FROM customers c
        JOIN orders o ON c.customer_id = o.customer_id
        JOIN order_payments op ON o.order_id = op.order_id
    WHERE op.payment_type = 'credit_card' AND op.payment_installments > 3
    GROUP BY c.customer_city
    HAVING SUM(op.payment_value) > 500000
) AS gasto
JOIN (
    SELECT c.customer_city, COUNT(DISTINCT oi.seller_id) AS vendedores_distintos
    FROM customers c
        JOIN orders o ON c.customer_id = o.customer_id
        JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.order_id IN (
        SELECT order_id
        FROM order_payments
        WHERE payment_type = 'credit_card' AND payment_installments > 3
    )
    GROUP BY c.customer_city
) AS vendedores
ON gasto.customer_city = vendedores.customer_city
ORDER BY gasto.gasto_total DESC;

```

■ Consulta difícil_2:

```

SELECT
    ct.product_category_name_english,
    COUNT(oi.order_item_id) AS cantidad_vendida,
    AVG(oi.price) AS precio_promedio
FROM (
    SELECT DISTINCT order_id
    FROM order_reviews
    WHERE review_score = 1
) r
JOIN order_items oi
    ON r.order_id = oi.order_id
JOIN products p
    ON oi.product_id = p.product_id
JOIN category_translation ct
    ON p.product_category_name = ct.product_category_name
GROUP BY ct.product_category_name_english
HAVING COUNT(oi.order_item_id) > (
    SELECT AVG(ventas_categoria)
    FROM (

```

```

SELECT
    COUNT(oi2.order_item_id) AS ventas_categoria
FROM (
    SELECT DISTINCT order_id
    FROM order_reviews
    WHERE review_score = 1
) r2
JOIN order_items oi2
    ON r2.order_id = oi2.order_id
JOIN products p2
    ON oi2.product_id = p2.product_id
JOIN category_translation ct2
    ON p2.product_category_name = ct2.product_category_name
GROUP BY ct2.product_category_name_english
) sub_promedio
)
ORDER BY cantidad_vendida DESC;

```

Parte 3

Cada consulta parametrizada tiene un puntaje de **15 puntos** (ambas suman 30 puntos). Por cada función se ejecutaron 15 test cases privados, 1 punto por cada correcto.

- Parte 3.1: A continuación se presenta una propuesta de solución:

```

def buscar_productos_por_presupuesto(db_path, categoria_ingles, precio_maximo):
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()

    query = """
        SELECT DISTINCT oi.product_id, oi.price, oi.seller_id
        FROM order_items AS oi
            JOIN products AS p ON oi.product_id = p.product_id
            JOIN category_translation AS ct ON p.product_category_name = ct.product_category_name
        WHERE ct.product_category_name_english = ?
            AND oi.price <= ?;
    """

    cursor.execute(query, (categoria_ingles, precio_maximo))
    resultados = cursor.fetchall()
    conn.close()

    return resultados

```

IMPORTANTE: La query interna de la función debe utilizar el operador DISTINCT. En caso de que el estudiante no lo haya utilizado pero la consulta de su función es semanticamente la misma, cada test case tiene 5/6 del puntaje correcto.

- Parte 3.2: A continuación se presenta una propuesta de solución:

```
def costo_envio_por_peso_y_ciudad(db_path, ciudad_destino, peso_minimo_gramos):
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()

    query = """
        SELECT oi.order_id, p.product_weight_g, oi.freight_value
        FROM order_items AS oi
            JOIN products AS p ON oi.product_id = p.product_id
            JOIN orders AS o ON oi.order_id = o.order_id
            JOIN customers AS c ON o.customer_id = c.customer_id
        WHERE c.customer_city = ?
            AND p.product_weight_g >= ?;
    """

    cursor.execute(query, (ciudad_destino.lower(), peso_minimo_gramos))
    resultados = cursor.fetchall()
    conn.close()

    return resultados
```

Parte 4

Cada consulta de la solución tiene un puntaje de **3 puntos** (ambas juntas suman 6 puntos). En caso de que una consulta use menos de cuatro operadores, la consulta llevará 0 puntos por no cumplir con los requisitos mínimos. Para cada consulta el puntaje se divide en dos partes:

- **(1.5 puntos) Interés de la consulta.** Se evalúa qué tan interesante es el resultado de la consulta según el dominio de los datos y qué tan relevantes son los operadores de SQL ocupados para lograrlo.
 - 1.5 puntos: interesante.
 - 1 punto: más o menos interesante.
 - 0.5 puntos: poco interesante.
 - 0 puntos: sin sentido / copia de una consulta del enunciado.
- **(1.5 puntos) Correctitud.** Se evalúa la correctitud de la consulta según lo propuesta en su declaración. Por cada error de correctitud se resta 0.5 puntos del punta total.
 - 1.5 puntos: consulta correcta sin errores.
 - 1 punto: la consulta tiene un error.
 - 0.5 puntos: la consulta tiene dos errores.
 - 0 puntos: la consulta tiene tres o más errores.

Si la consulta es evaluada como *sin sentido / copia de una consulta* en el ítem de **Interés de la consulta**, entonces será evaluada con 0 puntos en total, debido a que no es posible evaluar su correctitud.

Informe

Para esta parte de la tarea se evalúa la entrega del informe. El informe tiene un puntaje de **6 puntos** en total dividido de la siguiente manera por consulta:

1. **1.5 puntos** Consulta intermedia Parte 2.
2. **1.5 puntos** Consulta difícil Parte 2.
3. **1.5 puntos** Consulta 1 Parte 4.
4. **1.5 puntos** Consulta 2 Parte 4.

Para cada consulta anterior se evalúa la explicación, comprensión y presentación según el uso de SQL en el contexto del enunciado. El puntaje es discreto según la siguiente escala para cada consulta:

- **1.5 puntos:** Muy bien.
- **1.0 punto:** Satisfactorio.
- **0.5 puntos:** Deficiente.
- **0 puntos:** Incorrecto / inexistente.